

Joint Activation and Structured Sparsity for Speech Commands on Akida 1

Akida 1 Model Zoo

August 2026

Abstract

This report repeats, for a third time, the joint activation-sparsity and structured-sparsity investigation first conducted on Visual Wake Words (VWW) and then on PlantVillage, this time on Speech Commands keyword spotting (DS-CNN, audio rather than vision). The two prior studies disagreed sharply: on VWW the joint configuration was complementary (cheaper than the sum of its parts) at every setting tested; on PlantVillage an aggressive joint configuration was sharply antagonistic, though a lighter one avoided the problem entirely. This report finds a third pattern: at aggressive settings the joint configuration is complementary (1.84-point accuracy cost versus a naive additive prediction of 2.23 points), but at lighter settings it is slightly *super*-additive (0.92 points versus a predicted 0.64) — the sign of the interaction itself flips depending on where in the regularization-strength/prune-target space one looks. Unlike PlantVillage, neither joint operating point strictly dominates the other here: the aggressive configuration reaches more sparsity and a larger parameter reduction at twice the accuracy cost of the lighter one, a genuine two-point trade rather than a clear winner. Both remain good, usable operating points; the choice between them is a product decision, not a technical one.

Contents

1	Introduction	2
2	Background	2
2.1	Akida 1 and activation sparsity	2
2.2	Speech Commands keyword spotting	2
3	Dataset	3
4	Model architecture	3
4.1	A layer-naming issue specific to this model	4
5	Methodology	4
5.1	Activation sparsity: raw Hoyer-Square regularization	5
5.2	Structured sparsity: Network Slimming	5
5.3	Float-only evaluation	5
5.4	Training pipeline	5
6	Experimental design	6

7	Results	6
7.1	At aggressive settings, the joint configuration is complementary	7
7.2	At lighter settings, the interaction flips sign	7
7.3	Neither joint operating point dominates the other	8
7.4	Neither point resembles PlantVillage’s collapse	8
8	Discussion and limitations	8
9	Conclusion	8

1 Introduction

This is the third of three repeated studies investigating whether combining activation-sparsity regularization with structured (channel) pruning costs more, less, or the same as the sum of each mechanism’s individual accuracy cost. The first two studies, on VWW and PlantVillage, reached opposite conclusions:

- **VWW**: the joint configuration was complementary (sub-additive) at every setting tested, and a lighter joint configuration simply used the accuracy budget more efficiently – there was no trap to avoid, only a smooth curve to pick a point on.
- **PlantVillage**: an aggressive joint configuration was sharply antagonistic (super-additive by an order of magnitude), traced to a pruning-recovery problem; a lighter joint configuration avoided this almost entirely and was a strict win on every axis.

This report asks the same two questions as both prior studies – does the joint cost more or less than the sum of its parts, and can a better-optimized operating point be found – on Speech Commands (DS-CNN keyword spotting, 12 classes, MFCC audio features), a third model, dataset, and input modality entirely.

All work in this report is on the branch `akida1-sparsify-speech-commands-joint`, following up on the single-axis activation-sparsity experiment on branch `akida1-sparsify-speech-commands` (`SPARSITY_EXPERIMENT.md`) and on the joint VWW and PlantVillage studies.

2 Background

2.1 Akida 1 and activation sparsity

Akida 1 accelerates inference by skipping computation on zero-valued activations, so sparser intermediate activations run faster and at lower power. Structured (channel) pruning is a separate property – how much of the model’s parameter count has been physically removed – that Akida 1’s execution model does not exploit for speed; its value is reduced memory footprint.

2.2 Speech Commands keyword spotting

The task is 12-way audio classification: 10 keyword classes, plus a catch-all “Unknown” class and a “Silence” class, from one-second audio clips. Unlike VWW and PlantVillage (both image classification), inputs here are Mel-frequency cepstral coefficient (MFCC) features extracted from raw audio, giving this study its first non-vision data modality.

3 Dataset

The dataset is the `speech_commands` TensorFlow Datasets corpus, preprocessed into MFCC features matching the MLPerf Tiny keyword-spotting benchmark’s parameters.

Table 1: Dataset and feature-extraction summary, as implemented by `speech_commands_data_loader.py`.

Property	Value
Classes	12 (10 keywords + Silence + Unknown)
Raw audio	16 kHz, 1-second clips (padded/truncated)
Feature type	MFCC (10 coefficients) over 49 time frames
Feature shape	$49 \times 10 \times 1$
Split	train / test / validation (standard TFDS split)
Train-time augmentation	waveform time-shift, SpecAugment (frequency/time masking)

Features are quantized to `uint8` using percentile-based clipping bounds computed once from the training split (`compute_mfcc_range`).

4 Model architecture

The model is DS-CNN (`build_ds_cnn` in `speech_commands_model.py`): a depthwise-separable CNN keyword-spotting architecture, by far the smallest of the three models studied across all three reports (22,668 parameters, versus VWW’s 226,906 and PlantVillage’s 1,156,054). Like the other two, it is a strict feed-forward chain with no residual/skip connections. It consists of a stem convolution, three “plain” depthwise-separable blocks, a fourth depthwise-separable block ending in global average pooling, and a pointwise-convolution classifier head.

Table 2: DS-CNN architecture (production config: 64 filters, `pointwise_conv` classifier head). Every `conv2d/separable_conv2d` row is immediately followed by its own BatchNormalization and ReLU (omitted for brevity; parameter counts include them).

Layer	Type	Output shape	Params (layer + BN)
<code>input</code>	Input	$49 \times 10 \times 1$	0
<code>rescaling</code>	Rescaling ($\div 255$)	$49 \times 10 \times 1$	0
<code>conv2d</code>	Conv2D, stride 2	$25 \times 5 \times 64$	1,920
<code>dropout</code>	Dropout	$25 \times 5 \times 64$	0
<code>separable_conv2d</code>	SeparableConv2D	$25 \times 5 \times 64$	4,992
<code>separable_conv2d_1</code>	SeparableConv2D	$25 \times 5 \times 64$	4,992
<code>separable_conv2d_2</code>	SeparableConv2D	$25 \times 5 \times 64$	4,992
<code>separable_conv2d_3</code>	SeparableConv2D + global-avg-pool	64	4,992
<code>dropout_1</code>	Dropout	64	0
<code>reshape</code>	Reshape	$1 \times 1 \times 64$	0
<code>conv2d_1</code>	Conv2D (pointwise, logits)	$1 \times 1 \times 12$	780
<code>flatten</code>	Flatten	12	0
Total			22,668
Trainable / non-trainable			22,028 / 640

The classifier head here is a 1×1 pointwise convolution followed by `Flatten` (the production config’s `classifier_head: pointwise_conv` setting), rather than VWV’s/PlantVillage’s direct Dense layer – functionally equivalent for a model with no remaining spatial extent after global average pooling, but exercising a different code path in `speech_commands_model.py`.

4.1 A layer-naming issue specific to this model

Unlike VWV’s and PlantVillage’s pretrained-backbone loading path, which assigns every layer an explicit name, `build_ds_cnn` calls `akida_models.layer_blocks`’ block constructors without a `name=` argument. Keras auto-names unlabeled layers using a *per-process* global counter rather than resetting it per model, so building this model repeatedly within one Python process (once per grid point, as this sweep does) produces layer names that shift with each build: `separable_conv2d`, then `separable_conv2d_1`, ..., then on the next model `separable_conv2d_8`, and so on. Because the structured-sparsity implementation identifies prunable layers by name, the third grid point in this study’s initial run (the first one to actually prune) failed with `No such layer: separable_conv2d/BN`. The fix – calling `tf_keras.backend.clear_session()` immediately before each model build – resets Keras’s naming counter, so every model in the sweep gets the same, predictable layer names regardless of how many models the process has already constructed.

5 Methodology

The two sparsity mechanisms and the float-only rationale are identical to the VWV and PlantVillage studies; only the established activation-regularizer operating point differs.

5.1 Activation sparsity: raw Hoyer-Square regularization

An activity regularizer is attached to every ReLU layer’s output x :

$$\mathcal{L}_{\text{Hoyer}}(x) = \lambda \cdot \frac{(\sum_i |x_i|)^2}{\sum_i x_i^2 + \epsilon}. \quad (1)$$

Unlike VWW and PlantVillage, whose established best points both use the element-count-*normalized* variant, this project’s own single-axis experiment established $\lambda = 10^{-4}$ on the *raw* (unnormalized) form as the best point found: 71.3% activation sparsity (Akida-measured) for a ~ 3.1 -point accuracy cost. The normalized variant has not been separately validated for this model – this study reuses the established raw-form point rather than assuming normalization would help, consistent with this series’ general practice of not re-deriving what has already been validated.

5.2 Structured sparsity: Network Slimming

Network Slimming [1] is applied identically to the prior two studies, via `speech_commands_structured_sparsity.py` – a direct, unmodified port of the VWW implementation. **Pruning scope:** the three “plain” separable-conv blocks (`separable_conv2d`, `separable_conv2d_1`, `separable_conv2d_2`) are eligible; the stem and the final global-average-pooled block are excluded, since the latter feeds the classifier head directly.

5.3 Float-only evaluation

As with VWW and PlantVillage, no `cnn2snn quantize/QAT/convert` is used anywhere in this pipeline, for the same two reasons: it sidesteps the software-simulator-vs-hardware sparsity discrepancy documented in the single-axis experiments, and it keeps channel pruning decoupled from quantization concerns. Activation sparsity is the mean fraction of exactly-zero ReLU outputs on 1,000 held-out samples, and is **not directly comparable** to the single-axis experiment’s Akida-measured figures for the same reason as the other two studies.

5.4 Training pipeline

Unlike VWW and PlantVillage, which take training hyperparameters as command-line flags, this project is config-driven: model and training hyperparameters (filters, dropout, epochs, learning rate, augmentation) come from `configs/training_cfg.yml`, and only the regularizer strength/type and prune target are varied per grid point.

Phase 1 (float training, 30 epochs). Matches the production config’s float-training schedule (Adam, cosine-decay learning rate with 10% warmup, peak learning rate $\approx 4.8 \times 10^{-3}$).

Pruning (if $p > 0$). Channels pruned per Section 4.2, after a `clear_session()` reset (Section 4.1).

Phase 2 (post-prune fine-tuning, 15 epochs, learning rate 4×10^{-4}). Only run when $p > 0$, to recover accuracy lost to channel removal.

Training speed. This is by far the fastest model of the three studied: at 22,668 parameters and $49 \times 10 \times 1$ inputs, each epoch takes roughly 10–15 seconds on GPU 1 of the shared training server. The full pilot (4 configurations) completed in under 30 minutes, and the follow-up (3 more configurations) in under 25 minutes.

6 Experimental design

Pilot (2×2 grid). All four combinations of activation regularization on/off ($\lambda = 10^{-4}$, this project’s established best single-axis point) and pruning on/off ($p = 40\%$, a starting guess mirroring both prior studies’ choice on this axis).

Follow-up (lighter settings). Three lighter points, halving both knobs: $\lambda = 5 \times 10^{-5}$ and $p = 20\%$, individually and jointly.

Table 3: All configurations run. λ is the raw Hoyer-Square strength; p is the structured prune target.

Tag	λ	p
baseline	0	0%
activation_only	10^{-4}	0%
structured_only	0	40%
joint	10^{-4}	40%
activation_light	5×10^{-5}	0%
structured_light	0	20%
joint_light	5×10^{-5}	20%

7 Results

Table 4: Full results. Accuracy drop is relative to **baseline** (96.03%). Parameter reduction is relative to **baseline**’s 22,668 parameters. Activation sparsity is the float-model measurement and is *not* directly comparable to the single-axis experiment’s Akida-measured numbers.

Tag	λ	p	Accuracy	Acc. drop	Act. sparsity	Params (reduction)
baseline	0	0%	96.03%	–	64.7%	22,668 (–)
activation_only	10^{-4}	0%	95.14%	0.89pt	80.4%	22,668 (0%)
activation_light	5×10^{-5}	0%	95.41%	0.62pt	78.8%	22,668 (0%)
structured_only	0	40%	94.69%	1.34pt	65.0%	12,944 (42.9%)
structured_light	0	20%	96.01%	0.02pt	63.7%	17,468 (22.9%)
joint	10^{-4}	40%	94.19%	1.84pt	78.5%	12,944 (42.9%)
joint_light	5×10^{-5}	20%	95.11%	0.92pt	77.6%	17,468 (22.9%)

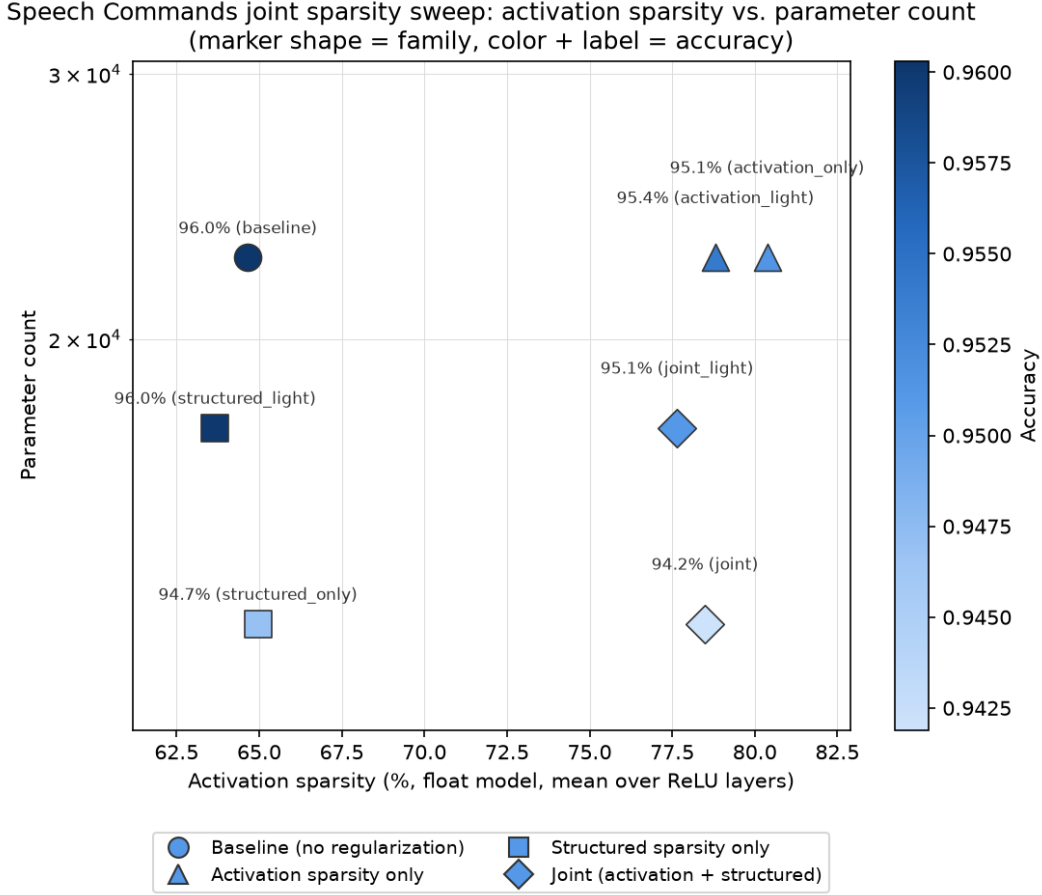


Figure 1: Activation sparsity vs. parameter count for all seven configurations. Marker shape encodes family (circle: baseline, triangle: activation sparsity only, square: structured sparsity only, diamond: joint); marker color and the accompanying label encode accuracy.

7.1 At aggressive settings, the joint configuration is complementary

Relative to **baseline**, **activation_only** alone costs 0.89 points and **structured_only** alone costs 1.34 points; a naive additive expectation predicts ~ 2.23 points for both together. The **joint** configuration actually costs **1.84 points** – less than the sum of its parts, while reaching 78.5% activation sparsity (close to **activation_only**’s own 80.4%) and the full 42.9% parameter reduction **structured_only** achieves alone. This matches the pattern VWV showed.

7.2 At lighter settings, the interaction flips sign

activation_light alone costs 0.62 points and **structured_light** alone costs a negligible 0.02 points; a naive sum predicts ~ 0.64 points. **joint_light** instead costs **0.92 points** – a modest but real 0.28-point excess over the naive prediction, the opposite direction from the aggressive corner’s result. Neither the VWV pattern (complementary throughout the range tested) nor the PlantVillage pattern (antagonistic specifically at aggressive settings, resolved by going lighter) fully describes this model: here the *sign* of the interaction itself depends on where in the (λ, p) space one looks.

7.3 Neither joint operating point dominates the other

Unlike PlantVillage, where the lighter joint configuration was a strict win on every axis, this model’s two joint points form a genuine two-point Pareto frontier: `joint` reaches more activation sparsity and a substantially larger parameter reduction (78.5% / 42.9%) than `joint_light` (77.6% / 22.9%), but at exactly double the accuracy cost (1.84pt vs. 0.92pt). Choosing between them is a product decision about which axis matters more, not a case where one point is simply better.

7.4 Neither point resembles PlantVillage’s collapse

Both joint configurations remain good, usable operating points – there is no sign here of the order-of-magnitude accuracy collapse PlantVillage’s aggressive joint setting produced. This model’s much smaller size (22,668 parameters and only 64 channels per prunable layer, versus VWV’s up to 256 and PlantVillage’s up to 512) may leave less room for the two regularizers’ pruning-relevant signals to diverge sharply – consistent with the single-axis experiment’s own observation that this model needed proportionally much larger activation-regularizer strengths to have an effect, itself a symptom of its small per-layer tensor sizes bounding the raw Hoyer-Square penalty’s magnitude.

8 Discussion and limitations

The interaction’s sign flip is not explained here. This report establishes that the joint interaction is sub-additive at the aggressive settings tested and super-additive at the lighter settings tested, but does not identify where the crossover occurs or why. Testing one or two intermediate points (e.g. $\lambda = 7 \times 10^{-5}$, $p = 30\%$) would help determine whether the transition is smooth or has its own structure.

Normalized Hoyer-Square remains unvalidated for this model. As noted in Section 4.1 and in the single-axis SPARSITY_EXPERIMENT.md, this project has never separately tested the normalized variant that both VWV and PlantVillage found to be their own best activation-sparsity mechanism. It is possible this study’s activation axis, and therefore its joint corners, would look different under that variant.

BN- γ regularization strength was not recalibrated. The $\mu = 10^{-5}$ BN-gamma L_1 strength driving channel selection was reused unchanged from the VWV/PlantVillage studies. Given this model’s much smaller per-layer channel counts (64, versus VWV’s 64–256 and PlantVillage’s 128–512), whether this strength is well matched to DS-CNN’s scale has not been established.

Float-vs-hardware sparsity gap and no latency validation. As with the other two studies, the float-model sparsity measurements here are not on the same footing as the single-axis experiment’s Akida-hardware/software-simulator-measured figures, and this report makes no claim about actual Akida 1 latency or power.

9 Conclusion

Repeating the joint activation-sparsity and structured-sparsity investigation a third time, on Speech Commands (DS-CNN, audio), produced yet a third distinct pattern from VWV and PlantVillage. At aggressive regularization/pruning strengths the joint configuration is complementary, matching

VWW’s finding; at lighter strengths it is instead mildly antagonistic, the opposite sign. Unlike PlantVillage, neither of the two joint operating points found here strictly dominates the other – `joint` trades more accuracy for more sparsity and a larger parameter cut than `joint_light`, a genuine two-point tradeoff rather than a clear-cut recommendation. Taken together across all three studies, the practical lesson is that the accuracy-vs-benefit interaction between activation-sparsity regularization and structured pruning is model- and regime-dependent – neither uniformly complementary nor uniformly antagonistic – and should be measured directly for each new model rather than assumed from a prior model’s result, even a closely related one.

References

- [1] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang. Learning Efficient Convolutional Networks through Network Slimming. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.
- [2] H. Yang, W. Wen, and H. Li. DeepHoyer: Learning Sparser Neural Network with Differentiable Scale-Invariant Sparsity Measures. In *International Conference on Learning Representations (ICLR)*, 2020.
- [3] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau, U. Thakker, A. Torrini, P. Warden, J. Cordaro, G. Di Guglielmo, J. Duarte, S. Gibellini, V. Parekh, H. Tran, N. Tran, N. Wenxu, and X. Xuesong. MLPerf Tiny Benchmark. *arXiv preprint arXiv:2106.07597*, 2021.